SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA04
COURSE NAME: Operating Systems

COURSE OUTCOME COVERED

CO1: *Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.*

QUESTIONS: 5 **Total Marks:100**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I Mounnila S P *(Name / ~~Reg No) certify that this submission is my~~ original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Mounnila

Signature of the student:

45

Annexure A

Short Answer Test

Question 1 (5 Marks)
Suppose a modern Operating System supports multiple users and processes with shared resources like CPU, memory, and I/O devices.
(a) Classify different types of Operating Systems suitable for this scenario and justify your choice.
(b) Draw the structure of an Operating System showing how resources are managed.
(c) Write a sequence of system calls required to create and execute a new process in such an environment. **(BL2)**

Question 2 (5 Marks)
Consider an Operating System managing multiple files and devices for concurrent users.
(a) Categorize system calls used for file and device management.
(b) Illustrate how the OS handles file operations using a layered structure diagram.
(c) Write a pseudo-code or C-based system call sequence to open, read, and close a file. **(BL3)**

Question 3 (5 Marks)
Assume a real-time Operating System used in an embedded system controlling industrial machinery.
(a) Identify the type of OS and explain why it is suitable for real-time applications.
(b) Draw a diagram representing the interaction between hardware, kernel, and user processes.
(c) List and explain system calls required for process synchronization in this system. **(BL2)**

Question 4 (5 Marks)
A distributed computing system consists of multiple interconnected computers sharing resources over a network.
(a) Classify the type of Operating System used and compare it with centralized OS.
(b) Draw the architecture of a distributed OS showing communication between nodes.
(c) Write the sequence of system calls involved in inter-process communication (IPC). **(BL3)**

Question 5 (5 Marks)
Consider a scenario where an Operating System must manage process scheduling, memory allocation, and device communication efficiently.
(a) Explain the major functions of an OS in handling these tasks.
(b) Draw a block diagram representing OS components and their interactions.

(c) Write system call examples for memory allocation and device management. **(BL3)**

RUBRICS FOR CALCULATION

Criteria	Weightage (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

OPERATING SYSTEMS — CO1 ASSESSMENT TOOL 1

Short Answers
Course Code: CSA0412

Name: Mounnila S P

Reg no: 192511188

Question 1 — Multi-user Operating System and Process Creation

(a) Types of Operating Systems and Justification: A multi-user, multitasking operating system is suitable because several users can access the system and run multiple processes while sharing CPU time, memory, and I/O devices. A time-sharing operating system gives each active process a small CPU time slice, improving responsiveness and allowing interactive use. A multiprogramming system keeps several jobs in memory so the CPU can work on another job while one job waits for I/O. Examples of multi-user operating systems include UNIX and Linux.

(b) Operating System Structure and Resource Management: The operating system provides an interface between users and hardware. Applications request operating-system services through the system-call interface. The kernel manages process scheduling, memory, files, device drivers, input/output operations, and security. It allocates CPU time through scheduling, assigns and protects memory, manages files, and controls devices through device drivers. The system-call interface lets applications request these services safely.

(c) Process Creation and Execution: In a UNIX-like operating system, fork() creates a child process. The child can use exec() to replace its process image with a new program. The parent may use wait() to wait for the child, and exit() terminates a process. The sequence is: fork() → child calls exec() → parent calls wait() → process terminates.

Question 2 — File and Device Management

(a) Categories of System Calls: File-management system calls include open(), read(), write(), close(), lseek(), and unlink(). They are used to create or open files, read and write data, reposition within a file, close files, and remove files. Device-management calls include open(), read(), write(), close(), and ioctl(). They provide access to devices, transfer data, and configure device-specific settings. Information-maintenance calls such as stat() and fstat() retrieve file or device status information.

(b) Layered Structure for File Operations: When an application requests a file operation, the request passes through the library or system-call interface to the kernel. The Virtual File System provides a common interface to different file systems. The file-system layer manages directories, permissions, and metadata. The block I/O subsystem and device driver then communicate with the storage device, such as an SSD or HDD. The result or error status is returned to the application.

(c) Opening, Reading, and Closing a File: A program uses open() to open a file and obtain a file descriptor. The read() call reads bytes from the file into a buffer. The program can then process or display the data. Finally, close() releases the file descriptor. Error checking is required after file operations to detect failures.

Question 3 — Real-Time Operating System and Synchronization

(a) Real-Time Operating System and Its Suitability: A Real-Time Operating System (RTOS) is suitable for an embedded system controlling industrial machinery because it is designed to respond to events within predictable time limits. In a hard real-time system, missing a deadline may cause system failure or unsafe operation. In a soft real-time system, occasional deadline misses reduce quality but may not cause total failure. Industrial safety-critical control commonly requires hard or tightly bounded real-time behavior.

(b) Interaction Between Hardware, Kernel, and User Processes: Control and monitoring processes request services from the RTOS kernel through system calls and events. Sensors and timers generate interrupts or events. The kernel handles interrupts, schedules tasks according to priority and timing requirements, and uses device drivers to communicate with hardware such as the CPU, sensors, actuators, timers, and I/O devices.

(c) System Calls and Process Synchronization: Synchronization mechanisms coordinate processes or threads and protect shared resources. sem_wait() and sem_post() wait for or release a semaphore. pthread_mutex_lock() and pthread_mutex_unlock() lock and unlock a mutex to protect a critical section. pthread_cond_wait() and pthread_cond_signal() allow a thread to wait for a condition and signal a waiting thread. mq_send() and mq_receive() transfer messages through a message queue. pthread_create() creates a thread, while pthread_join() waits for its completion. A mutex prevents simultaneous access to a shared resource, a semaphore coordinates access or signals events, and a message queue transfers data between tasks. Real-time systems must also consider priority inversion, bounded blocking, and deadlines.

Question 4 — Distributed Operating System and IPC

(a) Distributed Operating System and Centralized OS: A distributed operating system coordinates multiple interconnected computers and aims to make their resources available across the system. Nodes communicate over a network to exchange messages and access shared or remote resources. In a distributed system, processing and resources are spread across multiple networked nodes, and adding nodes can improve scalability and resource sharing. It must handle network communication and partial node failures. In a centralized operating system, processing and resource control are primarily handled by one central computer; expansion may depend on upgrading that machine, and central failure can make the main service unavailable.

(b) Distributed Operating System Architecture: Each node in a distributed system has its own processor, memory, operating-system services, and applications. The network enables nodes to exchange messages and request remote resources. Distributed coordination services help manage communication and resource access.

(c) Inter-Process Communication (IPC): Inter-process communication allows processes to exchange data. One common IPC method is a pipe between related processes. The pipe() call creates read and write ends, and fork() creates a child process. The sender uses write() to send data, while the receiver uses read() to receive it. The processes close unused pipe ends and release resources with close(); wait() or exit() is used to coordinate process completion. For unrelated processes or networked nodes, named pipes, message queues, shared memory, and sockets may be used.

Question 5 — OS Functions, Components, Memory and Devices

(a) Major Functions of an Operating System: Process management and scheduling create and terminate processes, select which process runs, switch the CPU between processes, and coordinate execution. Memory management tracks memory use, allocates and frees memory, provides virtual memory where supported, and protects address spaces. Device and I/O management use device drivers, handle interrupts, buffer data when needed, and coordinate input/output operations. File-system management organizes files and directories, manages metadata and permissions, and supports file access. Protection and security control access to system resources and separate user programs from privileged kernel operations.

(b) Operating System Components and Their Interactions: Applications request services through the system-call interface. The kernel coordinates process scheduling, memory management, the file system, I/O, protection, networking, and inter-process communication. Device drivers translate operating-system requests into device-specific operations. The operating system interacts with the CPU, main memory, disks, keyboard, network, display, and other devices.

(c) Memory Allocation and Device Management: On UNIX-like systems, mmap() maps memory into a process's address space, and munmap() removes the mapping. In C programs, malloc() and free() are common library functions for dynamic memory allocation and release, although they are not themselves direct system calls on typical POSIX systems. Device-management operations commonly use open(), read(), write(), close(), and ioctl(). The actual device path and supported operations depend on the operating system and device driver.

